

Object Oriented Programming

OOPS, Object Oriented Programming, is a way of organizing programs around objects. The four main concepts are Encapsulation, Inheritance, Polymorphism and Abstraction.

A good order to learn it in:

```
Class → Object → Constructor → Encapsulation → Inheritance  
      → Polymorphism → Abstraction → Interface
```

1. Class and object

A class is a blueprint. An object is an actual thing created from that blueprint.

```
class Car {  
    String brand;  
    int speed;  
  
    void drive() {  
        System.out.println(brand + " is driving");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Car car1 = new Car();  
        car1.brand = "Toyota";  
        car1.speed = 100;  
  
        car1.drive();  
    }  
}
```

Output:

```
Toyota is driving
```

Here `Car` is the class, and `car1` is an object of `Car`.

Think of it as:

```
Class   = blueprint of a house  
Object = the actual house
```

One class, many objects. Each object keeps its own copy of the fields.

2. Constructor

A constructor initializes an object when we create it. It has the same name as the class and no return type.

```
class Car {
    String brand;

    Car(String brand) {
        this.brand = brand;
    }
}

public class Main {
    public static void main(String[] args) {
        Car car1 = new Car("Toyota");
        Car car2 = new Car("BMW");

        System.out.println(car1.brand);
        System.out.println(car2.brand);
    }
}
```

Output:

```
Toyota
BMW
```

`this.brand` means the instance variable belonging to the current object. The plain `brand` on the right is the parameter.

If we write no constructor, Java gives us a default one that takes no arguments. As soon as we write our own, that default is gone.

3. Encapsulation

Encapsulation means hiding an object's internal data and controlling how it is accessed. Variables are made private, and methods provide the controlled way in.

```
class BankAccount {  
    private double balance;  
  
    public void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
        }  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

Now this is not allowed:

```
account.balance = -5000;    // Error: balance is private
```

Instead:

```
account.deposit(1000);
```

The check inside `deposit` cannot be skipped, so the balance can never be set to nonsense from outside.

So:

```
Encapsulation = data hiding + controlled access
```

4. Inheritance

Inheritance allows one class to reuse and extend another class.

```

class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking...");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();

        dog.eat();    // inherited from Animal
        dog.bark();   // its own
    }
}

```

Output:

```

Eating...
Barking...

```

The relationship is:

```

Animal
  ↑
Dog

```

A `Dog` is an `Animal`. That is why inheritance is called an IS-A relationship.

Java allows a class to extend one class only. It can implement many interfaces, which is covered in section 7.

5. Polymorphism

Polymorphism means one name can behave differently depending on the object involved.

The important case is method overriding.

```

class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Woof!");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Meow!");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a1 = new Dog();
        Animal a2 = new Cat();

        a1.sound();
        a2.sound();
    }
}

```

Output:

```

Woof!
Meow!

```

Both variables have type `Animal`, but Java calls the method belonging to the actual object. That is runtime polymorphism.

Java also supports method overloading: the same method name with different parameters.

```

void add(int a, int b) { }

void add(int a, int b, int c) { }

void add(double a, double b) { }

```

A useful distinction:

Overloading → compile-time polymorphism
Overriding → runtime polymorphism

6. Abstraction

Abstraction means showing what something does while hiding the implementation details.

For example, we drive a car using the steering wheel and the pedals without needing to know how the engine works.

Java supports abstraction with abstract classes and interfaces.

```
abstract class Vehicle {  
    abstract void start();  
  
    void stop() {  
        System.out.println("Vehicle stopped");  
    }  
}  
  
class Car extends Vehicle {  
    @Override  
    void start() {  
        System.out.println("Car started");  
    }  
}
```

We cannot create an object directly from an abstract class:

```
Vehicle v = new Vehicle();    // Error
```

But we can do this:

```
Vehicle v = new Car();  
v.start();
```

Output:

```
Car started
```

7. Interface

An interface defines a contract that implementing classes agree to follow.

```
interface Payment {  
    void pay(double amount);  
}  
  
class CreditCardPayment implements Payment {  
    @Override  
    public void pay(double amount) {  
        System.out.println("Paid " + amount + " by card");  
    }  
}
```

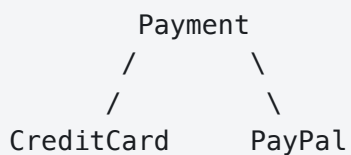
Usage:

```
Payment payment = new CreditCardPayment();  
payment.pay(500);
```

Output:

```
Paid 500.0 by card
```

This becomes useful in larger applications, because we can create several implementations:



The application works with `Payment` without caring which implementation it receives. Adding a third payment type adds a class and changes nothing in the calling code.

The four pillars together

Concept	Simple meaning	Java feature
Encapsulation	Protect data	<code>private</code> , getters and methods
Inheritance	Reuse and extend behavior	<code>extends</code>
Polymorphism	Same name, different behavior	overriding, overloading
Abstraction	Hide implementation details	<code>abstract</code> , <code>interface</code>

For example, imagine we are building a food delivery application. `Restaurant` , `Customer` , `Order` and `DeliveryPartner` would be classes. Each individual customer is an object. Customer details are protected through encapsulation. Different payment methods use polymorphism. A `Payment` interface provides abstraction, and specialized classes use inheritance.

The most useful thing at this point is not to memorize the definitions. Build a small Java program that uses all four.

Abstract class in depth

An abstract class is declared with the `abstract` keyword. It cannot be created directly, and it can hold both abstract methods, which have no body, and concrete methods, which do.

```
abstract class Animal {  
    abstract void makeSound();    // no implementation  
  
    void eat() {                  // has implementation  
        System.out.println("Eating...");  
    }  
}
```

This line:

```
abstract void makeSound();
```

says that every concrete subclass of `Animal` must supply its own `makeSound()`.

```

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Woof!");
    }
}

class Cat extends Animal {
    @Override
    void makeSound() {
        System.out.println("Meow!");
    }
}

```

Now:

```

Dog dog = new Dog();
dog.makeSound();
dog.eat();

Cat cat = new Cat();
cat.makeSound();
cat.eat();

```

Output:

```

Woof!
Eating...
Meow!
Eating...

```

Why use an abstract class. Every `Animal` should have `eat()`, but different animals make different sounds. The common behavior goes in the abstract class, and the specific behavior is forced onto the subclasses.

```

Animal
├── common behavior → eat()
└── required behavior → makeSound()
    ↓
    Dog → Woof
    Cat → Meow

```

Interface in depth

An interface is a contract that tells a class what it must do, but not how to do it.

```
interface Payment {  
    void pay(double amount);  
}
```

The interface says: any class that implements `Payment` must provide a `pay()` method.

```
class CreditCardPayment implements Payment {  
    @Override  
    public void pay(double amount) {  
        System.out.println("Paid using credit card");  
    }  
}  
  
class UpiPayment implements Payment {  
    @Override  
    public void pay(double amount) {  
        System.out.println("Paid using UPI");  
    }  
}
```

Now:

```
Payment payment1 = new CreditCardPayment();  
Payment payment2 = new UpiPayment();  
  
payment1.pay(1000);  
payment2.pay(1000);
```

Output:

```
Paid using credit card  
Paid using UPI
```

The application only needs to know about `Payment`. It does not need to care whether the real implementation is `CreditCardPayment`, `UPIPayment`, `PayPalPayment` or anything added later.

Abstract class vs interface

Abstract class	Interface
Represents a common base or type	Represents a contract or capability
Declared with <code>abstract class</code>	Declared with <code>interface</code>
A class uses <code>extends</code>	A class uses <code>implements</code>

Abstract class	Interface
Can have instance variables	Fields are implicitly <code>public static final</code>
Can have constructors	Cannot have constructors
Can have abstract methods	Can have abstract methods
Can have concrete methods	Can have <code>default</code> and <code>static</code> methods with bodies
A class can extend only one	A class can implement many

For example, a class extends an abstract class:

```
abstract class Animal {
    abstract void makeSound();

    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Woof");
    }
}
```

Whereas a class implements an interface:

```
interface Flyable {
    void fly();
}

class Bird implements Flyable {
    @Override
    public void fly() {
        System.out.println("Flying");
    }
}
```

The key conceptual difference:

```
Abstract class → what is it?
Interface      → what can it do?
```

A `Bird` is an `Animal`, and it can fly. The first is an abstract class, the second is an interface. That is usually enough to decide which one a new type needs.